SmartTAR STAR Fix 13 RC5 – Feature Summary

Core concept

SmartTAR STAR is a Windows GUI archiver built on top of the system tar.exe.
It creates a custom transparent container with the .star extension.

A .star file is an outer TAR container with this structure:

```
manifest.json
blocks/
  000001_structure.tar
  000002_stored.tar
  000003_compressible.tar.xz
```

The archive is not a black box. The outer .star container can also be inspected manually with tar.exe.

Main features

1. Transparent .star container

The archive uses the .star extension and contains:

- manifest.json
- blocks/

Advantages:

- readable structure,
- easier diagnostics,
- manual inspection is possible,
- block-based model instead of one opaque stream.

2. Block-based archive model

Data is not stored as a single monolithic archive, but as internal blocks.

Example in Hybrid mode:

```
000001_structure.tar
000002_stored.tar
000003_compressible.tar.xz
```

Each block has its own manifest entry:

- id
- group
- path

- compression
- method
- fileCount
- sourceBytes
- sizeBytes
- sha256
- reason

3. manifest.json

Every archive contains a metadata manifest.

The manifest contains, for example:

- format
- toolVersion
- createdUtc
- compressionMode
- sourceName
- sourceType
- sourceBytes
- rootRule
- creationMode
- planning
- capabilities
- sourceProfile
- groupStageDiagnostics
- blocks

This allows the archive to describe:

- how the archive was created,
- which source was archived,
- which blocks are included,
- which compression method each block uses,
- which SHA-256 hashes belong to each block,
- whether group-stage succeeded or RC6 fallback was used.

4. Root-preserving extraction

The archive stores the original source root name using:

- sourceName
- sourceType

During extraction, the .star file name is not authoritative. The manifest
decides the extraction root.

Example:

Archive file name:

My_backup.star

Manifest contains:
sourceName = "XviD4PSP 5"

Extraction creates:
XviD4PSP 5
Renaming the .star file does not change the extraction root.

Compression modes

1. Hybrid

Default mode.

Typical data mapping:

```
compressible -> XZ9
diskimage    -> ZSTD19
stored       -> STORE
```

Practical example:

```
000001_structure.tar
000002_stored.tar
000003_compressible.tar.xz
```

Recommended as the general-purpose mode.

2. Smart

More detailed grouping by data type:

- text
- binary
- executable
- diskimage
- media
- archives
- unknown

Example mapping:

```
text       -> XZ
binary     -> ZSTD
executable -> ZSTD
diskimage  -> ZSTD
media      -> STORE
archives   -> STORE
unknown    -> XZ
```

3. Solid

Automatically chooses one main compression method based on the source profile.
Uses one group:

solid

4. Smart XZ

Uses XZ for most groups except data that usually should not be recompressed.

5. Store

Creates TAR blocks without compression.

Useful for:

- already compressed data,
- fast packing,
- diagnostics.

Supported compression methods
=============================
SmartTAR tests the capabilities of the available tar.exe.

Supported methods:

- STORE
- GZIP
- BZIP2
- XZ
- XZ9
- ZSTD19

Current main methods:

XZ9   -> compressible/text-like data
ZSTD19 -> binary/executable/disk-image-like data
STORE   -> media/archive-like data

Group-stage architecture
========================

Main change compared with RC6

Older RC6 builds created multiple chunk blocks:

compressible_p001
compressible_p002
...
compressible_p031

The RC5 branch prefers:

compressible -> one group-stage block

For example:

000003_compressible.tar.xz

Advantages:

- fewer blocks,
- smaller manifest,
- less TAR overhead,
- better compression,
- faster verification,
- cleaner archive structure.

How group-stage works

For each data group, SmartTAR creates a temporary stage folder.

Files are not copied into the stage by default. They are linked through hardlinks:

original file -> hardlink inside stage

Then tar.exe is called as:

tar.exe ... -C stage .

Advantages:

- tar.exe does not receive a long file argument list,
- tar.exe does not receive a file list,
- tar.exe works with a simple safe stage path.

Hardlink stage
==========

No double I/O

Group-stage primarily uses hardlinks.

This means:

- no full copy of the data is created,

- no second content cache is created,
- tar.exe reads the same physical file through the stage path.

Advantages:

- faster than copying,
- more storage-efficient,
- suitable for larger archives.

Literal hardlink paths

RC4 fixed issues with file names such as:

[01].py

PowerShell can interpret [and] as wildcard characters.

The current RC5 build contains the function:

New-HardLinkLiteral

This function:

- uses Test-Path -LiteralPath,
- escapes wildcard characters,
- creates directories through .NET,
- if New-Item -ItemType HardLink fails, it tries a fallback through:

cmd.exe /c mklink /H

This helps with names such as:

- [01].py
- file (copy).txt
- paths with spaces
- paths with diacritics

RC6 fallback

If group-stage fails, SmartTAR does not fail the entire archive creation.

It uses emergency fallback:

RC6 chunked block creation

For example:

compressible_p001
compressible_p002
...

Fallback should now be rare, but it is still useful as a safety mechanism.

It may happen when:

- a file is deleted during archiving,
- a file is locked,
- the filesystem does not support hardlinks,
- source and workroot are not on the same volume,
- tar.exe fails on an edge case.

Group-stage diagnostics

RC5 writes the following section into the manifest and reports:

Group-stage diagnostics:

Example:

compressible: group-stage-ok, files=2915, source=402.18 MB, message=Created as one RC5 group-stage block. XZ directory timestamps normalized.

Fallback example:

compressible: fallback-rc6-chunked, message=Group-stage failed. ...

This makes it clear:

- which group succeeded,
- which group used fallback,
- why fallback was used,
- how many files were in the group,
- how large the source data for the group was.

SHA-256 verification

Each internal block has a SHA-256 hash.

The manifest contains:

sha256

During VERIFY, SmartTAR checks:

- whether each block exists,
- whether each block can be listed with tar -tf,
- whether each block SHA-256 hash matches.

The report then shows:

Archive verification OK
Blocks OK: ...

Blocks failed: ...

VERIFY mode

The GUI contains a button:

VERIFY

This verifies the archive without extracting it.

It checks:

- outer .star container,
- manifest.json,
- blocks,
- SHA-256 hashes,
- whether TAR blocks can be listed.

The verification report shows:

- Format
- Tool
- Version
- Mode
- Creation mode
- Blocks
- Blocks OK
- Blocks failed
- Archive size
- Group-stage diagnostics
- OK/FAIL block list

Salvage mode

The GUI contains the option:

Salvage mode (Ignore broken blocks)

During extraction, this mode allows damaged blocks to be skipped and readable parts of the archive to be recovered.

Salvage was more important when archives contained many chunk blocks. With group-stage architecture it is less central, but still useful.

Example use case:

stored block is intact
compressible block is damaged

It is also useful when an archive was created through RC6 fallback with multiple chunk blocks.

Extraction safety

Path validation

During extraction, SmartTAR checks that paths inside blocks are safe.

Blocks are listed through:

```
tar -tf
```

SmartTAR checks for:

- no absolute paths,
- no paths like C:\...,
- no .. segments,
- no path traversal attempts.

Overwrite warning

Before extraction, SmartTAR reads the manifest and determines the real target root.

If the target folder already exists, SmartTAR shows a warning:

```
Target already exists.
Existing files/folders may be merged or overwritten.
Continue?
```

The user can choose:

```
Yes -> continue
No  -> cancel extraction
```

The check is based on sourceName from the manifest, not on the .star file name.

XZ deterministic stage metadata

RC5 adds targeted stabilization for XZ blocks.

The issue was that repeatedly packing the same source could produce slightly different .tar.xz sizes because newly created stage directories had current timestamps.

RC5 sets timestamps only for directories inside XZ stages to a stable value:

```
2000-01-01 00:00:00
```

Files are not modified.

Applies only to:

- XZ
- XZ9

Does not apply to:

- STORE
- GZIP
- BZIP2
- ZSTD

The report then shows:

XZ directory timestamps normalized.

File timestamp preservation

Because SmartTAR uses hardlinks:

original file -> hardlink inside stage

file metadata, especially LastWriteTime, should be preserved through the TAR layer.

XZ compression does not remove file timestamps because:

.tar.xz = TAR metadata + XZ compression

Realistically expected to be preserved:

- content,
- path,
- size,
- LastWriteTime / modification time.

Not guaranteed as full NTFS backup metadata:

- CreationTime,
- LastAccessTime,
- ACL,
- owner,
- alternate data streams,
- reparse points.

Temporary folder cleanup

SmartTAR creates a safe working directory:

SmartTAR_Temp

```
  smarttar_create_xxx
  smarttar_extract_xxx
  smarttar_verify_xxx

After completion it removes:

smarttar_* work folder

If the root folder is empty, it also removes:

SmartTAR_Temp

Cleanup uses retry logic and a fallback through:

cmd.exe /c rmdir /s /q


Reports
=======
After operations, SmartTAR creates text reports.

Examples:

archive.star.create_report.20260605_...
archive.star.verify_report.20260605_...
archive.star.extract_report.20260605_...

The report contains:

- Source size
- Archive size
- Ratio
- Saved
- Verify status
- Blocks
- Group-stage diagnostics
- OK/FAIL blocks


Practical results after RC5
===========================
On a test where earlier builds created many blocks:

RC2/RC3:
Blocks: 33
Archive size: 149.06 MB

After RC4/RC5:
Blocks: 3
Archive size: about 141.89 MB

On a smaller test:
```

Source size: 102.60 MB
Archive size: 13.75 MB
Blocks: 2
compressible: group-stage-ok
XZ directory timestamps normalized

One-sentence summary
====================
SmartTAR STAR RC5 is a transparent block-based Windows archiver built on tar.exe. It creates .star containers with a manifest, SHA-256 verification, smart data grouping, XZ/ZSTD/STORE compression based on content type, group-stage hardlink architecture, RC6 fallback, salvage extraction, safe root-preserving extraction, and targeted XZ stage metadata stabilization.

Current status:

RC6 = last stable old chunk architecture
RC4 = first truly successful group-stage architecture
RC5 = current best candidate for the new stable branch